

Research Copilot — Interview Prep

Detailed Answers — RAG, Agentic Systems & Production LLM Engineering

Full explanations for all 50 questions, referencing this project's actual ADRs, real ablation numbers, and real bugs found and fixed during the eval-harness build. Use these to check your own answers, not as a substitute for attempting the questions cold first.

Topics to Learn

Study these before attempting the questions. Each maps directly to a section below and to real decisions/bugs from this project.

Vector Databases & ANN Search	HNSW, approximate vs. exact nearest-neighbor, quantization, sharding, Qdrant vs. pgvector vs. Weaviate vs. Pinecone trade-offs.
Embedding Models	Bi-encoders vs. cross-encoders, asymmetric instruction prefixes, domain mismatch, dimensionality trade-offs, MTEB benchmarking.
Hybrid Search & Fusion	BM25/sparse retrieval, dense retrieval, Reciprocal Rank Fusion (RRF), why ranks (not raw scores) are fused.
Chunking Strategies	Recursive/structure-aware chunking, chunk-size tuning, semantic chunking, Parent-Child chunking, Contextual Retrieval, Late Chunking.
Reranking	Cross-encoder rerankers, why they outperform bi-encoders at small scale, LLM-as-reranker, domain mismatch in rerankers.
Agentic Loops & Tool Calling	Hand-rolled vs. framework-based (LangGraph) agent loops, native function calling, iteration caps, conversation state, malformed tool calls.
Citation, Faithfulness & Hallucination Mitigation	Structured-output citation schemas, deterministic verification, post-hoc NLI/embedding attribution, RAGAS faithfulness vs. context precision/recall, abstention design.
RAG Evaluation Design	Golden sets, hand-labeling vs. synthetic labels, precision@k/recall@k/MRR, ablation studies, adversarial/partial-coverage test design.
LLM API Reliability Engineering	Rate limits vs. daily quotas, retry/backoff strategy design, timeout handling, silent model-alias regressions, defensive error handling.

Cost & Latency Trade-offs	Per-call latency compounding in multi-call agent loops, model selection (thinking vs. lite models), cost-at-scale estimation.
Financial Document Domain Knowledge	Inline XBRL, SEC filing structure, restatement/version drift, why dense financial filings stress-test RAG systems differently than prose.
System Design & Architecture Decision-Making	ADR-driven decision documentation, incremental build philosophy, scaling bottleneck analysis, cross-domain architecture reuse.

Contents

A. Vector Databases & Retrieval Architecture

A1, A2, A3, A4, A5, A6

B. Embedding Models

B1, B2, B3, B4, B5

C. Hybrid Search & Fusion

C1, C2, C3

D. Chunking Strategies

D1, D2, D3, D4, D5, D6

E. Reranking

E1, E2, E3, E4

F. Agentic Loops & Tool Calling

F1, F2, F3, F4, F5

G. Citation, Faithfulness & Hallucination Mitigation

G1, G2, G3, G4, G5

H. RAG Evaluation Design

H1, H2, H3, H4, H5, H6

I. LLM API Reliability Engineering

I1, I2, I3, I4, I5

J. Cost & Latency Trade-offs

J1, J2, J3

K. Financial Document Domain Knowledge

K1, K2, K3

L. Capstone System Design Scenarios

L1, L2, L3, L4

A. Vector Databases & Retrieval Architecture

A1. You're building a RAG system that needs to scale from 100K to 50M chunks over 18 months, on a team of 2 engineers with no dedicated DevOps, and a \$500/month infra budget. You initially chose Qdrant self-hosted via Docker on a single VM. At what point would you need to change your architecture, and what would you change first: the vector store, the hosting model, or something else? Justify with specific technical triggers, not just "when it gets slow."

Don't reach for a new vector store first — that's the least likely bottleneck early on. The actual technical triggers, roughly in the order you'd hit them: (1) Memory pressure: HNSW indexes are held largely in RAM for query speed; at some point (often single-digit millions of vectors at typical embedding dimensions) your single VM's RAM becomes the binding constraint before Qdrant's query logic does. The first real fix is enabling scalar or product quantization (a config change, not a re-architecture) to shrink the memory footprint, often 4x, at a small recall cost you can measure against your own eval harness. (2) Single point of failure / no HA: a single VM means any restart or crash is full downtime — at this scale that's an operational risk before it's a performance one. The fix here is a hosting-model change (managed Qdrant Cloud, or a small multi-node cluster), not a vector-store swap. (3) True horizontal scale beyond one machine's RAM even after quantization — only at this point do you seriously consider sharding across multiple Qdrant nodes (Qdrant supports this natively) or, if your team's operational capacity genuinely can't sustain a distributed system with 2 engineers, evaluate a managed vector DB service. The team-size and budget constraints in the prompt are the real signal here: with 2 engineers and no DevOps, you should exhaust every configuration-level lever (quantization, connection pooling, read replicas) before taking on the operational complexity of a distributed system — that trade-off, not "50M is a big number," is what should drive the decision.

A2. Contrast Qdrant, pgvector, Weaviate, and Pinecone across: operational ownership, hybrid search support, and learning value for an engineer trying to deeply understand retrieval internals. In what specific scenario would pgvector actually outperform a dedicated vector DB, not just "be simpler"?

Qdrant: self-hosted or managed, native hybrid search (dense+sparse with RRF fusion) via named vectors, transparent REST/gRPC API — highest learning value because you implement the fusion logic yourself and see every step. pgvector: runs inside Postgres you likely already operate, so zero additional infrastructure; hybrid search is possible via combining a vector column with Postgres full-text search (tsvector), but it's bolted-on rather than native, and ANN performance/tooling is less mature than a dedicated engine at scale. Weaviate: self-hosted, most "batteries included" (built-in vectorization and reranking modules), but its GraphQL API and module abstractions hide the mechanics you're trying to learn. Pinecone: fully managed SaaS, zero infrastructure ownership, but a complete black box — you get an API, not an understanding of HNSW tuning, quantization, or fusion internals. The genuine scenario where pgvector outperforms a dedicated vector DB isn't about raw query speed — it's about TOTAL SYSTEM COMPLEXITY at modest scale. If your vectors and their relational metadata (user permissions, timestamps, business entities) need to be joined in the same query constantly, and your data volume is in the low millions of vectors, keeping everything in one Postgres instance avoids a dual-write consistency problem (keeping two systems in sync) that a separate vector DB introduces. The "win" is architectural simplicity and transactional consistency, not performance — pgvector wins by

having one fewer moving part when your access pattern is fundamentally relational-plus- vector, not when you need the fastest possible ANN search.

A3. Your Qdrant collection uses HNSW indexing. Explain, in your own words, why HNSW is approximate and not exact nearest-neighbor search, and describe one concrete scenario in this project (SEC filings retrieval) where that approximation could cause a wrong answer to surface without any bug in your code.

HNSW (Hierarchical Navigable Small World graphs) builds a multi-layer graph where each vector is a node connected to a small number of "nearby" neighbors, and search proceeds by greedily walking the graph from an entry point toward the query vector, descending through layers of decreasing sparsity. This is fast (logarithmic-ish in practice) because it never compares the query against every vector in the collection — but that's exactly why it's approximate: the greedy graph walk can get stuck in a locally-good-but-not-globally-best region and terminate before finding the true nearest neighbor, especially in high-dimensional space where "nearby" is a fuzzier concept. Concretely in this project: two chunks from different banks' 10-Ks could both be near-duplicates of the same boilerplate risk-factor language (many SEC filings share templated legal text), sitting close together in embedding space. If the TRUE best match for a query is a chunk that HNSW's graph walk doesn't traverse to (because the walk terminated at a "good enough" neighboring cluster first), a slightly-less-relevant chunk could surface instead — with zero bug in your code, purely a property of approximate search trading recall for speed. This is why an eval harness measuring Hit@k against real ground truth matters: it's the only way to detect this class of failure, since it's invisible from code review alone.

A4. You've been asked to add scalar quantization to reduce Qdrant's memory footprint by 4x. What accuracy/recall tradeoff should you expect, and how would you validate whether that tradeoff is acceptable for THIS project specifically, using infrastructure you already built?

Scalar quantization compresses each float32 vector component down to int8 (a 4x memory reduction), at the cost of some precision in distance calculations — typically a small but non-zero recall drop, often in the low single-digit percentage range, though the exact number is corpus- and model-dependent, which is exactly why you shouldn't accept a vendor's rule of thumb without measuring it yourself. The right validation approach for this project specifically: re-run the existing retrieval ablation (`eval/retrieval_ablation.py`) against a quantized collection using the SAME 28-question ground truth already built, and compare Hit@1/Hit@5/MRR against the unquantized baseline (32.1%/53.6% at the current chunk size, from the real ablation results). If Hit@5 drops by, say, 2-3 percentage points, that's likely an acceptable trade for a 4x memory reduction; if it drops by 15+ points, quantization is hurting more than the memory savings justify at this corpus size, and you'd hold off until scale actually demands it. The key discipline is: don't estimate this trade-off theoretically — you already have the exact harness needed to measure it directly against your own data and your own questions, which is more trustworthy than any published benchmark.

A5. Your vector store is showing p99 query latency of 800ms at 2M vectors, well above your 200ms SLA. Walk through your diagnostic process: what's the first thing you check, second, third? Where does sharding fit into that list, and where does it NOT belong?

First: separate the hybrid pipeline into its components and time each independently — embedding the query (local model inference time), sparse encoding, the Qdrant fusion query itself, and reranking. A p99 dominated by local CPU model inference (embedding/reranking) points to a completely different fix (batching, GPU, or a faster model) than a p99 dominated by the Qdrant query itself. Second: if the vector store's own query time is the culprit, check whether it's compute-bound (CPU saturation on the Qdrant host, visible via basic monitoring) or memory-bound (index doesn't fit in RAM, causing disk swaps — a much larger latency cliff, not a gradual slowdown). Third: check whether p99 (not p50) specifically means a small number of queries are pathological — e.g., unusually long query text, or queries hitting a specific under-indexed segment — rather than uniform slowness across all queries, since these need very different fixes (query-side validation/truncation vs. infrastructure scaling). Sharding belongs LATE in this list, and only once you've confirmed the bottleneck is genuinely compute/memory capacity on a single node that quantization and hardware upgrades can't resolve — sharding adds real complexity (query fan-out, result merging, more nodes to operate) and doesn't fix a problem that's actually rooted in one slow embedding call per query, a memory-bound single node that could be fixed with quantization, or a small number of pathological queries. Reaching for sharding before ruling those out is a common mistake: distributing a bottleneck that isn't actually about total data volume just adds operational surface area without fixing the root cause.

A6. Why did the project keep vectors/metadata in Qdrant, separate from a relational database, rather than using pgvector to have "one source of truth" in Postgres? What operational cost does that separation introduce that isn't obvious until you've actually built ingestion pipelines for both structures?

ADR-001 chose Qdrant specifically to maximize learning depth on dedicated vector-DB mechanics (HNSW tuning, native hybrid fusion, quantization) over pgvector's operational simplicity — a deliberate trade given this project's explicit goal of learning retrieval internals, not minimizing moving parts. The operational cost that isn't obvious until you've actually built the ingestion pipeline: every document that gets chunked and embedded now has TWO places its lifecycle needs to be tracked — the chunk's existence/content in Qdrant, and (in a fuller production system) any relational metadata about that document (ownership, access permissions, ingestion timestamps, source document versioning) that would live in a separate relational store. If a document is deleted or updated, you now need a coordinated update across two systems rather than one transaction — and if that coordination logic has a bug or a partial failure (Qdrant update succeeds, relational update fails, or vice versa), you get silent drift between "what the vector store thinks exists" and "what the system of record thinks exists." This project didn't build a separate relational metadata store (payload metadata lives directly in Qdrant's point payloads), so this specific cost hasn't been paid yet — but it's exactly the cost ADR-001 flagged as a consequence of the Qdrant choice, and it's the concrete "when would you revisit this" trigger: the moment document metadata needs relational querying (e.g., "show me all documents a specific user uploaded"), that's when the dual-system consistency problem becomes real.

B. Embedding Models

B1. Explain why BGE-family embedding models require an instruction prefix ("Represent this sentence for searching relevant passages:") for QUERIES but not for documents. What class of embedding models does NOT have this asymmetry, and what's the architectural reason?

BGE and similar "asymmetric" embedding models are trained with contrastive learning where queries and passages are deliberately different distributions of text (a query is often a short question; a passage is a longer, declarative chunk of prose) — the instruction prefix acts as a signal telling the model "treat this input as a search query, not as a document," steering it toward the region of embedding space where queries live, which was calibrated during training specifically to align well with the (unprefixed) embedding space of passages. Without the prefix, a genuine question and a declarative passage answering it can end up further apart in embedding space than the training process intended, silently hurting retrieval without any error being raised — exactly the kind of implementation detail that's easy to get wrong. Models that don't have this asymmetry are typically "symmetric" or general-purpose sentence embedding models (e.g., a plain sentence-transformers model trained via general-purpose sentence-similarity objectives) that treat queries and documents identically because their training data doesn't distinguish between the two roles — often at some cost in retrieval-specific accuracy compared to a model deliberately trained for the query/passage asymmetry.

B2. You swap your embedding model from bge-base-en-v1.5 (768-dim) to a new model with 1536 dimensions that scores higher on MTEB. List every downstream component in a hybrid-search RAG pipeline (like this project's) that would break or need re-work, beyond "just re-embed everything."

(1) The Qdrant collection's ``dense`` vector field has a fixed ``size`` parameter set at collection-creation time (``VectorParams(size=dense_dim, ...)`` in this project's ``vector_store.py``) — a dimension change requires creating a NEW collection, not just re-upserting into the existing one, since Qdrant won't accept mismatched-dimension vectors into an existing field. (2) Every already-ingested chunk needs full re-embedding and re-upsert — there's no partial-migration path, since old vectors are dimensionally incompatible with new queries. (3) The reranker (a separate cross-encoder model) is independent of embedding dimensionality and does NOT need to change — a detail worth stating explicitly in an interview, since it shows you understand which components are actually coupled to embedding dimension and which aren't. (4) Any code hardcoding the old dimension (this project's ``EmbeddingProvider.dimension`` property is used by ``QdrantHybridStore.__init__`` to size the collection) needs updating, but since it's already a property read dynamically from the provider rather than hardcoded, this specific project's code handles it cleanly — an example of why exposing ``dimension`` as an interface property rather than a magic number matters. (5) Your retrieval ablation's ground truth chunk_ids remain valid (they're keyed by source file + chunk index, independent of embedding), but Hit@k/MRR NUMBERS need to be re-measured from scratch since the new model changes retrieval behavior — you cannot assume MTEB's general benchmark improvement transfers to your specific corpus and question set without re-running your own eval.

B3. Gemini's free-tier embedding API was wired in as a configurable alternative to a local BGE model. If asked "why not just use the API embedding model as the default since it's higher quality," what's your answer, and what assumption in that question is wrong?

The assumption baked into the question is that "higher quality on paper" should automatically outweigh operational cost — but the actual determining factor for a DEFAULT (not an available option) is: what do you need to do repeatedly and cheaply? During development, chunking strategy gets iterated on constantly (chunk size, overlap, structure), and EVERY iteration requires re-embedding the entire corpus. A free-tier API embedding model, even a good one, has real rate limits and (as this project discovered) surprisingly harsh DAILY quotas — that's incompatible with rapid iteration, since you'd routinely exhaust your daily embedding budget mid-experiment, days before you're anywhere near comparing models on quality. A local model has zero such limit: this project's chunk-size ablation alone required re-embedding an 18-file corpus twice at two different chunk sizes, which would have consumed a meaningful fraction of a typical free daily API quota in a single afternoon of legitimate experimentation. The API model remains available as a CONFIGURABLE alternative specifically for the one-time cost/quality comparison this project's cost model calls for — that's a fundamentally different usage pattern (one embedding pass, once) than what the default needs to support (unlimited iteration).

B4. What's the actual difference between a bi-encoder and a cross-encoder at the architecture level, not just "one is faster"? Why can't you use a cross-encoder for the initial retrieval step over millions of documents, even if you had unlimited compute?

A bi-encoder independently encodes the query and each candidate document into fixed-size vectors, and relevance is computed afterward as a similarity function (cosine, dot product) between two ALREADY-COMPUTED, independent embeddings — critically, this means every document's embedding can be precomputed once, stored, and reused for every future query. A cross-encoder instead concatenates the query and a specific candidate document TOGETHER as a single input, and passes that joint input through the transformer, letting every token in the query attend directly to every token in the candidate document via self-attention — this joint processing is what makes cross-encoders more accurate (the model can reason about query-document interaction directly, not just compare two independent summaries) but it also means the score is only defined for one specific (query, document) pair at a time and cannot be precomputed, since the query isn't known until inference time. Even with unlimited COMPUTE, this is an architectural, not a resource, limitation: to rank a query against a million documents with a cross-encoder, you'd need to run a million separate forward passes at query time (no precomputation possible), whereas a bi-encoder only needs one forward pass for the query, then a cheap similarity computation against millions of pre-computed vectors. This is exactly why the standard pattern (and this project's architecture) uses a bi-encoder for broad initial retrieval, then a cross-encoder reranker only on the much smaller top-k candidate set where the per-pair cost becomes affordable.

B5. Your embedding model was trained predominantly on web text and general documents. You're now embedding SEC 10-K filings full of financial jargon, ticker symbols, and regulatory boilerplate. What specific failure modes would you expect from a domain mismatch, and what are two ways to address it that DON'T involve fine-tuning your own embedding model?

Expected failure modes: (1) out-of-distribution vocabulary (specialized terms like "CET1 ratio," "stress capital buffer," ticker symbols) may get embedded less discriminatively, since the model has seen these patterns rarely during general-purpose training, causing genuinely different financial concepts to cluster closer together in embedding space than they should. (2) Structurally repetitive boilerplate (this

project found the SAME sentence template — "X totaled \$Y billion at December 31, 2023, compared to \$Z billion..." — reused across many different financial metrics within one filing) can confuse a general-purpose embedding model into treating structurally-similar-but-semantically-different sentences as more similar than they are, since the model wasn't specifically trained to distinguish domain-specific templated language at that level of granularity. Two fixes that don't require fine-tuning: (a) lean harder on hybrid search — this project's real ablation results show BM25/sparse retrieval catching exact keyword/entity matches (ticker symbols, exact dollar figures, defined terms) that dense embeddings alone under-weight, which is precisely the failure mode domain mismatch produces; combining both compensates for each other's blind spots. (b) invest in chunk-size and chunking-strategy tuning specifically for the domain — this project's empirical finding that smaller chunks (200 vs. 500 tokens) dramatically improved retrieval on this exact corpus is a chunking-level (not embedding-level) mitigation for a related symptom: isolating facts more cleanly reduces how much the embedding needs to disambiguate within a single chunk, partially compensating for the embedding model's reduced domain-specific discriminative power.

C. Hybrid Search & Fusion

C1. Explain Reciprocal Rank Fusion (RRF) mathematically, not just "it combines rankings," and explain why RRF uses RANKS rather than raw similarity SCORES to combine dense and sparse results. What problem would arise if you tried to combine raw cosine similarity and raw BM25 scores directly by weighted sum?

RRF assigns each document a fusion score of sum over each ranked list it appears in of $1/(k + \text{rank})$, where rank is the document's position in that list (1-indexed) and k is a small constant (commonly 60) that dampens the influence of very high ranks and prevents a single list's rank-1 item from completely dominating. Documents are then re-sorted by this summed score. The key reason RRF operates on RANKS rather than raw scores: cosine similarity (typically bounded in $[-1, 1]$ or $[0, 1]$ depending on normalization) and BM25 scores (an unbounded, corpus-and-query-length- dependent statistic with a completely different scale and distribution) are NOT comparable numbers — a BM25 score of 8.3 and a cosine similarity of 0.83 have no principled way to be weighted-summed together, since their magnitudes, distributions, and even their meaning (BM25 grows with query term frequency and corpus rarity; cosine similarity is a bounded angular measure) are fundamentally different. If you tried a naive weighted sum of raw scores, you'd either need to normalize both scores into some shared range first (introducing an arbitrary, corpus-dependent normalization choice that can itself distort results — e.g., min-max normalizing BM25 scores per-query means the same absolute relevance can score very differently depending on what OTHER candidates happened to be retrieved for that specific query), or you'd get results dominated by whichever score happens to have a larger raw numeric range for a given query, with no principled control over the balance. RRF sidesteps this entirely by discarding the score's magnitude and just using ordinal position — a document ranked #1 by BM25 contributes the same fixed $1/(k+1)$ regardless of whether its raw BM25 score was 5 or 50, making the fusion robust to the two systems' incompatible scales without requiring any per-query calibration.

C2. In this project's retrieval ablation, Hit@5 went dense-only 14.3% → +hybrid 28.6% → +reranking 32.1% (at the original 500-token chunk size). Explain, mechanistically, why BM25/sparse search would catch cases that dense embedding search misses, using a concrete example from a financial-filing corpus.

Dense embeddings excel at SEMANTIC similarity — capturing that two differently-worded sentences mean roughly the same thing — but they can under-weight exact, low-frequency tokens that carry disproportionate importance in a specific query, because the embedding compresses the whole text into a fixed-size vector that necessarily blends and dilutes individual token signals. BM25, by contrast, is a term-frequency/inverse-document-frequency statistic that rewards EXACT matches on rare terms heavily — it doesn't understand meaning at all, but it's extremely good at "this specific string appears here and it's rare across the corpus, so it's probably important." Concretely: a query like "What was ZION's total deposits" — the ticker symbol "ZION" is a short, specific, low-frequency token that a dense embedding model (trained mostly on general prose, not financial ticker conventions) might not weight as heavily as its semantic content ("total deposits," a common financial concept appearing in every bank's filing). BM25 would strongly favor documents containing the exact string "ZION" over documents that are merely semantically similar (e.g., mentioning "Zions Bancorporation" in prose without the ticker, or discussing a different bank's deposits using similar language) — precisely the kind of exact-entity matching where sparse retrieval outperforms dense retrieval, and precisely why the real ablation numbers in this project show hybrid search meaningfully outperforming dense-only search rather than the two being redundant.

C3. If your sparse (BM25) component and your dense component disagree strongly and consistently on the same class of queries, what would that tell you about your corpus or your queries, and how would you diagnose which one is "more right" without just picking a favorite?

Consistent disagreement on a specific query CLASS (not random, isolated disagreements) is a signal that the two retrieval mechanisms are being stressed by different properties of that query class — e.g., queries containing exact identifiers (ticker symbols, dollar figures, defined legal terms) will systematically favor BM25's exact-match strength, while queries phrased as paraphrased or conceptual questions (asking about "risk" broadly rather than a specific defined "Risk Factors" section) will systematically favor dense embedding's semantic generalization. Rather than picking a favorite, the right diagnostic is empirical: pull the SPECIFIC ground-truth chunk for several queries in the disagreeing class (this project already has hand-verified chunk_ids for exactly this purpose) and check which retrieval mode's top results actually contain it. If BM25 consistently wins on this class, that's evidence the class is dominated by exact-entity lookups (a legitimate strength to lean into, not a bug), whereas if dense wins, it's evidence of paraphrase-heavy or conceptual queries needing semantic generalization. This diagnosis directly informs whether RRF's fusion weighting (or, in some implementations, a configurable balance between dense/sparse contribution) should be tuned differently for different query types — rather than assuming one blend fits every query class equally, which the presence of a systematic disagreement pattern already disproves.

D. Chunking Strategies

D1. This project found that reducing chunk size from 500 tokens to 200 tokens raised Hit@5 from 32.1% to 53.6%. Explain the actual mechanism: why does a larger chunk size hurt embedding-based retrieval, even when the correct answer is fully contained within the larger chunk?

A chunk's dense embedding is a single fixed-size vector computed by pooling (typically mean- or [CLS]-token pooling) over the ENTIRE chunk's token representations — it is not a per-sentence or per-fact representation, it's one blended summary of everything in the chunk. When a chunk spans multiple, only loosely related topics (this project found real examples: a 1,400-2,000 character chunk opening with unrelated content like "recent volatility in the banking industry... capital and liquidity actions" before finally containing the actual target fact — "insured deposit ratio strengthened from 45% to 73%" — 80% of the way through), the pooled embedding is dominated by whatever content is more voluminous or more central to the chunk's overall "gist," diluting the signal for any single minority fact within it. So even though the correct answer is textually present in the chunk, the chunk's VECTOR REPRESENTATION doesn't strongly represent that specific fact — a query about the insured deposit ratio computes a lower similarity to this chunk's embedding than it would to a smaller, more topically-focused chunk containing ONLY that fact, because in the larger chunk the fact is a minority contributor to an embedding that's mostly "about" something else. This is a structural property of mean/pooled embeddings, not a retrieval algorithm bug — smaller, more topically coherent chunks produce embeddings that more purely represent their content, which is exactly what the empirical Hit@5 improvement (32.1% → 53.6%) confirmed when tested directly against real ground truth rather than assumed theoretically.

D2. You just cut chunk size from 500 to 200 tokens to fix a retrieval-precision problem, and it worked. Six weeks later, your citation-synthesis quality has degraded — the model is generating oddly fragmented, incomplete-sounding answers. What's the likely mechanism connecting your chunking change to this new problem, and what's the standard architectural fix?

Smaller chunks solve the retrieval-precision problem (isolating facts cleanly for embedding matching) but introduce a NEW problem at the synthesis stage: a 200-token chunk that precisely matches a query may lack the surrounding context needed for the LLM to write a complete, well-grounded answer — e.g., retrieving a chunk containing just "45% to 73%" without the sentence that identifies WHAT metric that's referring to, if the chunking boundary happened to split the identifying clause into an adjacent chunk. This is the classic precision/context tension in chunking: smaller chunks improve MATCHING precision but can starve the GENERATION step of sufficient surrounding context, producing exactly the "fragmented, incomplete-sounding" symptom described. The standard architectural fix is Parent-Child chunking (documented as an ADR-003 upgrade layer in this project, though not yet implemented): index small child chunks for precise embedding-based MATCHING, but when a child chunk is retrieved, expand to return its larger parent chunk/section as the actual context passed to the LLM for synthesis. This decouples the two competing needs — precision at match time, sufficiency at generation time — rather than forcing one chunk size to serve both jobs simultaneously.

D3. Compare Parent-Child chunking and Contextual Retrieval as two different fixes for the SAME underlying problem (facts losing meaning when isolated into small chunks). What is that shared underlying problem, and why are these two fixes NOT redundant with each other?

The shared underlying problem: isolating a chunk from its surrounding document strips away context a reader (human or LLM) needs to correctly interpret it — a chunk saying "it grew 20% that year" is meaningless without knowing what "it" and "that year" refer to, and a small, precisely-matched chunk is MORE likely to suffer from this than a larger one, creating tension with the precision benefits of small chunks (see D1/D2). Parent-Child chunking solves this at RETRIEVAL time: it matches on small, precise child chunks, but once a match is found, swaps in a larger parent chunk/section for what's actually passed downstream — the fix operates on what content gets SHOWN to the LLM. Contextual Retrieval (Anthropic's technique) solves a related but distinct instance of the same problem at INGESTION time: before embedding, an LLM generates a short contextual summary situating each chunk within the document (e.g., "this chunk discusses WAL's Q4 2023 deposit stabilization"), and that summary is prepended to the chunk before it's embedded — the fix operates on what content gets EMBEDDED, improving match quality itself, not just what's shown after a match. They're not redundant because they intervene at different pipeline stages solving different sub-problems: Contextual Retrieval improves the EMBEDDING'S ability to be found by a relevant query in the first place (a retrieval-quality fix), while Parent-Child chunking improves what the LLM SEES once a chunk has already been found (a generation-quality fix) — you could reasonably deploy both simultaneously, since Contextual Retrieval doesn't guarantee the retrieved chunk alone contains sufficient context for generation, and Parent-Child chunking doesn't improve the underlying embedding's semantic precision.

D4. Why was Late Chunking (Jina AI's technique) rejected for this project specifically, and what would have to change about the project's existing architecture for Late Chunking to become viable?

Late Chunking works by running the ENTIRE document through a long-context embedding model first (computing token-level embeddings that already incorporate full-document context via self-attention across the whole document), and only AFTER that full-document pass does it pool sub-ranges of those context-aware token embeddings into individual chunk vectors — meaning each chunk's embedding inherently "knows about" the whole document, solving the referent-loss problem (D3's "it grew 20% that year") at the embedding architecture level rather than via LLM-generated context. This was rejected specifically because it requires an embedding model that exposes PRE-POOLING TOKEN-LEVEL embeddings and supports long-context input — this project's chosen embedding model (ADR-002: local `bge-base-en-v1.5`) does not expose this; it only returns a single pooled sentence/passage embedding, with no API to access intermediate token representations. Adopting Late Chunking would require replacing the embedding model entirely (e.g., with a Jina embeddings model designed for this), which would directly undo ADR-002's rationale (free, unlimited local iteration with a well-benchmarked general-purpose model) for a technique whose real-world benefit over the cheaper alternative, Contextual Retrieval, isn't yet well-established. The concrete trigger for revisiting this: if this project ever adopted a long-context embedding model with exposed token embeddings for OTHER reasons (e.g., a future project phase needing longer-context retrieval generally), Late Chunking would become viable as a side benefit — but switching models solely to enable it isn't currently justified.

D5. You're chunking a corpus of technical API documentation (short, code-heavy pages with lots of headers) versus a corpus of financial 10-K filings (long, dense paragraphs mixed with huge financial tables). Would you use the same chunk_size and overlap parameters for both? Justify chunk size specifically in terms of what kind of "fact isolation" failure each corpus is prone to.

No — these corpora have structurally opposite failure modes. API documentation is typically already well-segmented by headers and short code blocks; the natural document structure often ALREADY approximates good chunk boundaries, so a larger chunk size risks little dilution (each header-delimited section is usually already about one topic) and might even be preferable to avoid splitting a code example or parameter list mid-way, which would make either half unusable on its own. Financial 10-K filings, as this project found directly, pack MULTIPLE distinct facts into dense, templated prose with weak internal structure (a single "Executive Summary" bullet list mixing total deposits, total loans, stockholders' equity, and nonperforming assets in adjacent bullets) — here, a larger chunk size actively causes the fact-dilution problem documented in D1, because the chunk boundary doesn't naturally align with "one self-contained idea" the way a documentation page's headers usually do. The general principle: chunk size should be tuned to how densely a corpus PACKS distinct, independently-queryable facts into its natural prose units, not treated as a fixed default — a corpus where facts are naturally isolated by structure tolerates larger chunks, while a corpus where many facts share dense, structurally similar paragraphs (SEC filings' repeated "\$X billion...compared to \$Y billion" template across many different metrics) needs smaller chunks specifically to keep those facts from diluting each other's embeddings, exactly as this project's real ablation numbers (32.1% → 53.6% Hit@5) demonstrated empirically rather than assumed from general principle alone.

D6. In this project, the same "Total deposits of \$X billion, up \$Y billion from December 31, 2022" sentence structure appeared in MULTIPLE unrelated contexts across a single 10-K (total deposits, total assets, total loans, stockholders' equity, all using nearly identical phrasing). Why does this specific pattern hurt BOTH sparse (BM25) and dense retrieval simultaneously, rather than being a problem for only one of them?

This pattern is a rare case where BOTH retrieval mechanisms are stressed by the same root cause, just via different mechanisms. For BM25/sparse retrieval: the shared boilerplate phrasing ("totaled," "compared to," "billion," "December 31, 2023," "2022") means many DIFFERENT facts share a large overlapping vocabulary of common, structurally-repeated terms — BM25's term-frequency/inverse-document-frequency weighting downweights common terms across the corpus, but if this exact templated phrasing appears dozens of times across one filing (once per financial metric), the SPECIFIC distinguishing term (deposits vs. assets vs. loans vs. equity) becomes a small fraction of the query's matchable vocabulary relative to the shared boilerplate, diluting the sparse signal's ability to discriminate between these near-identical-looking sentences. For dense retrieval: the same structural template produces embeddings that cluster closely together in vector space, since a transformer-based embedding model is sensitive to surface-level phrasing and sentence structure, not just abstract meaning — four sentences that are 90% identical in wording but differ in one key noun (deposits/assets/loans/equity) can embed as more similar to EACH OTHER than a truly semantically-precise model "should" produce, since the shared structural pattern dominates the embedding's overall representation. This is exactly why this specific failure mode (structurally-templated, high-lexical- overlap boilerplate across financially-distinct facts) is one of the harder retrieval problems in this domain — it isn't solved simply by combining dense+sparse (hybrid), since both are independently degraded by the same underlying corpus property, which is why chunk-size reduction (isolating each templated sentence into its own smaller, less-crowded chunk) was the fix that actually moved the needle, rather than fusion-method tuning.

E. Reranking

E1. Explain, at an architectural level, why a cross-encoder reranker can afford to be much more accurate than the bi-encoder used for initial retrieval, given that both are "just" transformer models scoring relevance.

The accuracy difference comes from WHEN and HOW the query and document interact within the model, not from one architecture being inherently "smarter." A bi-encoder computes the query's embedding and the document's embedding completely independently — the two representations never influence each other during encoding, and relevance is only computed afterward via a simple similarity function (cosine, dot product) on the two already-finished vectors. A cross-encoder instead feeds the query and document TOGETHER into the transformer as one joint sequence, allowing full self-attention between every query token and every document token during processing — the model can directly learn interactions like "does this specific document token satisfy this specific query token's need," something a bi-encoder architecturally cannot do since by the time similarity is computed, both vectors are already finalized and can't be adjusted based on each other. This joint attention is computationally more expensive (as explained in B4, it can't be precomputed per-document since the pairing is query-specific) but produces meaningfully more accurate relevance judgments precisely because it has access to fine-grained, pairwise interaction information the bi-encoder's independently-computed, already-compressed vectors have thrown away. The trade-off is fundamentally compute-for-accuracy: the reranker "affords" to be more accurate because it's only run on a small candidate set (this project's RETRIEVE_K=20 → top-5 after reranking), where the per-pair cost is manageable, unlike the millions of candidates the initial bi-encoder retrieval must handle.

E2. At 500-token chunks, adding a reranker improved Hit@5 from 28.6% to 32.1% — a clear win. At 200-token chunks, reranking improved Hit@5 (46.4% → 53.6%) but WORSENE MRR (0.354 → 0.320) and Hit@1 (28.6% → 21.4%) compared to hybrid search alone. Propose two different hypotheses for why a reranker's benefit would change character when chunk size changes, and describe an experiment to distinguish between them.

Hypothesis 1 — the reranker is domain-mismatched, and smaller chunks expose this more starkly: the cross-encoder reranker (`bge-reranker-base`) was trained on general-purpose relevance data, not financial-filing text. At larger chunk sizes, hybrid retrieval's top-20 candidates might already be fairly noisy (containing many marginally-relevant chunks due to the dilution problem from D1), giving the reranker more room to add value by filtering out genuinely poor candidates. At smaller, better-isolated chunks, hybrid retrieval's candidates are ALREADY fairly precise (the correct chunk is often ranked highly by hybrid alone, per the strong 200-token hybrid-only MRR of 0.354), so the reranker has less "obviously bad" content to filter and instead starts making marginal, domain-uninformed re-orderings among already-good candidates — sometimes correctly promoting a genuinely better match into the top-5 (helping Hit@5), but sometimes bumping the single BEST match down from rank 1 to rank 2-3 based on the reranker's own imperfect, domain-mismatched judgment (hurting Hit@1/MRR specifically). Hypothesis 2 — this is an artifact of RETRIEVE_K staying fixed at 20 while chunk granularity changed: with smaller chunks, the same 20-candidate window covers LESS of the

document's total content per candidate, potentially changing which true positives are even present in the reranker's input set to begin with, independent of the reranker's own judgment quality. To distinguish between these: run the reranker in isolation on a FIXED, hand-inspected set of top-20 hybrid candidates for several specific chunk-200 queries, and manually compare the reranker's chosen top-5 against the ground truth chunk directly — if the correct chunk is present in the candidate set but the reranker demotes it (supporting Hypothesis 1, a judgment-quality issue), that implicates the reranker's own scoring; if the correct chunk is ABSENT from the candidate set entirely for the misranked queries (supporting Hypothesis 2), that implicates the earlier retrieval stage's coverage, not the reranker's judgment.

E3. Why was LLM-as-reranker explicitly excluded from the production runtime path in this project, even though it was evaluated as an ablation? What's the actual bottleneck, and how does that bottleneck change for a low-QPS internal tool instead of a consumer product?

The bottleneck is query-time LATENCY AND COST compounding across candidates, not raw capability — a dedicated cross-encoder reranker scores all N candidates efficiently in essentially one batched forward pass, while an LLM-as-reranker approach requires either N separate LLM calls (one per candidate) or one large prompt containing all N candidates for the LLM to rank — both are dramatically slower and more expensive per query than a purpose-built reranker model, especially when this cost is paid on EVERY user request rather than once at ingestion time (unlike, say, Contextual Retrieval's one-time ingestion-time LLM cost). ADR-004 explicitly reserved LLM-as-reranker for a ONE-TIME offline ablation (comparing its accuracy against the dedicated cross-encoder) rather than a runtime path, precisely because the question "does an LLM reranker outperform a dedicated one" is worth answering once with real numbers, not worth paying for on every request. This calculus genuinely changes for a low-QPS internal tool: if a system serves, say, 50 queries a day for a small internal team rather than thousands of concurrent consumer requests, the aggregate cost and latency of LLM-as-reranker becomes far more tolerable in absolute terms — a few extra seconds and a few cents per query might be entirely acceptable when total query volume is low, especially if the LLM's more nuanced, instructable judgment (e.g., "penalize outdated information," which a generic cross-encoder can't be instructed to do) provides real value. The right framing for an interview: latency/cost-per-query concerns don't disappear at low QPS, but the THRESHOLD for "is this acceptable" shifts substantially based on total request volume and the value of per-query customization.

E4. If your reranker is trained on general web-relevance data and you deploy it on a financial-filings domain, what specific failure mode would you expect at inference time, and how would you detect it using only the metrics your eval harness already computes, without adding new instrumentation?

Expected failure mode: the reranker's learned notion of "relevance" was calibrated on web-search-style query/document pairs (short queries, varied document types, general topics) — when applied to financial-filing text with its own dense jargon, heavily templated phrasing, and domain-specific notions of what makes one passage more relevant than another (e.g., understanding that "period-end" vs. "average" deposits are meaningfully different framings of a similar-sounding fact, a distinction this project encountered directly in the eval-ground-truth-building process), the reranker may fail to distinguish between subtly-different-but-critically- distinct financial statements, or may over-weight

superficial lexical overlap with the query in a way a domain-aware model wouldn't. Concretely, this is the "reranker demotes the actually-correct chunk" scenario discussed in E2's Hypothesis 1. To detect this WITHOUT new instrumentation: compare the ablation's own Hit@1 and MRR (rank-sensitive metrics, sensitive to whether the BEST match is at the very top) against Hit@5 (a looser, "is it anywhere in the top 5" metric) across the with-reranker vs. without-reranker configurations you already compute. A domain-mismatched reranker's signature is exactly what this project observed: Hit@5 improving (the correct chunk is still somewhere in a broader window) while Hit@1/MRR degrade (the reranker isn't confident enough, or is actively wrong, about WHICH single candidate is best) — this specific pattern, visible purely from metrics you already have without adding any new logging or evaluation code, is a strong signal of reranker judgment quality issues rather than a retrieval-coverage problem.

F. Agentic Loops & Tool Calling

F1. Explain the difference between "automatic function calling" and a hand-rolled agent loop. What do you lose in terms of learning/control by using automatic function calling, and what do you gain in terms of engineering time?

Automatic function calling (offered by some SDKs, including google-generativeai's ``enable_automatic_function_calling`` option) handles the entire tool-calling loop internally: the SDK detects a function call in the model's response, executes the corresponding Python function automatically, feeds the result back to the model, and repeats — all inside a single library call, with no application code managing the loop's state machine. A hand-rolled agent loop (this project's approach, ADR-005) implements each of those steps explicitly: inspecting the response for tool calls, executing them, constructing the tool-result message, appending it to conversation history, deciding when to stop (iteration cap, or the model declining further tool calls), and handling malformed responses defensively (this project hit exactly this in practice — a tool call missing its expected argument, requiring a manual fallback the automatic path wouldn't have exposed you to reasoning about). What you lose in automatic mode: a concrete, hands-on understanding of the state machine itself — you don't see how conversation history needs to be threaded, how to structure a termination condition, or how to defensively handle a model's imperfect tool-call compliance, since the SDK does all of that invisibly. What you gain: significantly less boilerplate code and faster initial implementation, since you don't need to write or debug the loop mechanics yourself — a reasonable trade for a straightforward, single-tool use case in a production setting where the mechanics are well-understood and reliability matters more than the learning exercise, but a worse trade when the explicit goal (as in this project) is to genuinely understand what a framework or automatic mode abstracts away.

F2. Your hand-rolled agent loop has `MAX_SEARCH_ITERATIONS=4` as a hard cap. A user asks a genuinely complex multi-hop question that would benefit from 6 search iterations to fully answer. What are three different design responses to this situation, and what are the tradeoffs of each?

(1) Simply raise the cap (e.g., to 6 or higher): the simplest fix, but it increases worst-case latency and cost linearly for EVERY question, even simple ones that only need 1 iteration, since the cap is a ceiling

not a target — this doesn't harm simple questions directly (the loop still stops early once the model signals it's done), but it does increase the WORST-CASE tail latency and cost exposure, which matters if you have any SLA or budget constraint on individual requests. (2) Detect when the cap is hit and surface that explicitly to the user/downstream synthesis step (e.g., "search budget exhausted, answering with partial evidence") rather than silently truncating: this preserves cost predictability while being honest about reduced confidence — the citation-verification pass this project built would ideally treat a cap-truncated answer differently from a naturally-completed one, since the risk of an incomplete-evidence answer is different from a well-supported one, even if both technically produced "an answer." (3) Make the cap adaptive/dynamic based on question complexity signals (e.g., detecting multi-part questions, or letting the model itself request additional iterations with justification) rather than a fixed constant: this is the most sophisticated approach and best matches genuine per-question need, but introduces real complexity — you now need a reliable way to detect "this question needs more iterations" BEFORE running out, which itself requires either heuristics that can be wrong or an additional model call to assess complexity, adding its own latency/cost. The right choice depends on whether your dominant cost driver is worst-case latency (favor a fixed conservative cap with explicit truncation signaling) or answer completeness on complex queries (favor adaptive limits, accepting the added complexity).

F3. Why does this project's agent loop maintain conversation HISTORY across search iterations rather than treating each search call as a stateless, independent request? What would break if you made it stateless?

The agent loop's entire value proposition is letting the MODEL decide whether/what/when to search — that decision-making process inherently depends on the model remembering what it has already searched for and what it learned from previous searches, so it can decide "have I gathered enough evidence yet, or do I need another, differently-phrased query." If each search call were stateless (the model given no memory of prior searches), you'd lose the ability for the model to reason about GAPS in what it's already found — it couldn't recognize "I searched for X and got a partial answer, now I need to search for the related fact Y that would complete the picture," since it would have no memory that it already searched for X or what it found. Concretely, this project's multi-hop questions (e.g., tracing a company's deposits across multiple filing periods) specifically depend on the model being able to look at what it already retrieved and decide a NEXT, differently-targeted query is needed — a stateless design would force the model to either guess all needed queries upfront in one shot (defeating the point of an iterative agentic loop) or would never be able to build on partial progress across iterations. This is precisely why ``chat.start_chat(history=[])`` and repeated ``chat.send_message()`` calls (which implicitly accumulate conversation history within the SDK's chat session object) are used rather than independent, memoryless ``generate_content()`` calls per search iteration.

F4. At what point does a single-tool agent loop justify migrating to a graph-based framework like LangGraph? Name three concrete signals in your own codebase that would tell you "now is the time," not just "frameworks are generally good."

(1) Multiple tools with interdependent, non-linear routing: if this project added a second tool (e.g., a numeric calculator for computing derived financial metrics, alongside ``search``) and the model's choice of WHICH tool to call next depended on branching logic more complex than "call search until satisfied, then answer" — e.g., "if search returns ambiguous results, ask a clarifying sub-question; if it returns a

number, verify it with the calculator tool" — that branching complexity is exactly what a graph-based state machine is designed to express cleanly, versus a growing tangle of nested if/else logic in a hand-rolled loop. (2) Need for human-in-the-loop approval gates: this project's own portfolio roadmap (Project 2, the support-agent copilot) explicitly requires pausing execution for human review before certain high-risk actions — that's a distinct control-flow pattern (suspend, wait for external input, resume) that a simple loop with an iteration counter doesn't naturally support, but that LangGraph's checkpointing/state-persistence features are built for. (3) Needing to REPLAY or DEBUG a specific point in a multi-step execution: once an agent's execution graph gets complex enough that you need to inspect or resume from an intermediate state (rather than just re-running the whole loop from scratch), the state-management overhead a hand-rolled loop requires you to build yourself starts to genuinely duplicate what a framework offers for free — at that point, the "framework abstracts away learning" cost from F1 is outweighed by the real engineering cost of re-implementing checkpointing/state-persistence yourself.

F5. During testing, the model called the search tool without providing the required query argument, causing an unhandled crash. Beyond the specific defensive fix, what does this incident tell you about a broader principle for building production systems on top of LLM tool-calling, regardless of provider?

The broader principle: LLM tool-calling, even with a strictly-defined JSON schema marking an argument as `required`, is not a hard guarantee the way a statically-typed function signature is — the schema is a STRONG HINT to the model, not an enforced contract the way a compiler enforces function signatures in traditional software. Any code consuming LLM-generated structured output (tool calls, JSON responses, function arguments) needs to treat that output as fundamentally UNTRUSTED INPUT, the same way you'd treat user-submitted form data or an external API's response — validate, defensively fall back, and never assume required fields are actually present just because you specified them as required in a schema. This isn't a provider-specific quirk to patch around once; it's a category of bug that will recur with any LLM tool-calling integration, across any provider or model, precisely because the "compliance" is probabilistic (the model was trained to usually follow the schema) rather than mechanically enforced. The practical takeaway for production systems: every consumer of LLM structured output should have an explicit fallback/default path for missing or malformed fields (as this project's fix demonstrates — `call.args.get("query", question)` rather than `call.args["query"]`), and every eval/test harness exercising LLM tool-calling should be resilient to individual malformed responses (this project's second real fix — wrapping per-question evaluation in a broad exception handler) rather than treating a single malformed response as a fatal, whole-run-ending error.

G. Citation, Faithfulness & Hallucination Mitigation

G1. Explain the difference between: (a) prompting the model to self-report inline citations, (b) structured output with a per-claim citation schema plus deterministic verification, (c) post-hoc NLI/embedding-based attribution after free-form generation. For each, name the specific failure mode it does NOT protect against.

(a) Self-reported inline citations (e.g., "answer with [1], [2] markers"): cheapest to implement, but has NO enforcement mechanism — the model can place a plausible-looking citation marker next to a claim that the cited source doesn't actually support, and nothing in the pipeline catches this, since there's no verification step at all. Failure mode NOT protected against: confidently-wrong attribution (citing a real, retrieved chunk that simply doesn't support the specific claim next to it). (b) Structured output with per-claim schema + deterministic verification (this project's approach, ADR-006): forces the model to explicitly name which chunk IDs support each claim, and a deterministic check confirms every cited ID actually exists in the retrieved evidence set — this catches the case where a citation references a NONEXISTENT or non-retrieved chunk (a stronger guarantee than (a)), but it does NOT verify that the cited chunk actually ENTAILS the specific claim text — a claim could cite a real, retrieved chunk ID that happens to be topically related but doesn't actually substantiate the specific assertion made, and the deterministic "does this ID exist" check would pass anyway. (c) Post-hoc NLI/embedding attribution: generates freely first, then separately checks whether each output sentence is actually ENTAILED by some retrieved chunk (using a natural-language- inference model or embedding similarity as a proxy for entailment) — this protects against exactly what (b) misses (weak/false entailment to a real citation), but introduces its own failure mode: NLI models themselves aren't perfect at judging entailment, particularly for numeric or highly domain-specific claims (financial figures, precise legal language), so a "verified" claim under this method could still be a false positive if the NLI model's own judgment is wrong, and it adds a full extra pipeline stage with its own latency/cost and potential to introduce new categories of error rather than eliminating error entirely.

G2. Your citation-verification eval found that for two adversarial questions, the system produced a confident, cited-looking answer instead of correctly abstaining. Both questions concerned data that existed ONLY as an image in the source document. Walk through the most likely failure point: retrieval, synthesis, or verification? How would you instrument the pipeline to find out definitively?

All three stages are plausible failure points, and distinguishing between them requires inspecting INTERMEDIATE state, not just the final output. If the failure is in RETRIEVAL: the agent's search queries for "deposit composition" or "gender composition" likely still return SOME chunks from the source document (even if none of them actually contain the specific data point, since that data lives only in an unindexed image) — text elsewhere in the filing that's topically adjacent (e.g., general prose about deposits, without the specific percentage breakdown) could be retrieved and mistaken by the LLM as sufficient evidence. If the failure is in SYNTHESIS: even given genuinely irrelevant/insufficient retrieved evidence, the LLM might still generate a claim with a citation to one of those retrieved-but-insufficient chunks, essentially "trying to be helpful" by extrapolating or guessing at a plausible-sounding number not actually supported by the cited text. If the failure is in VERIFICATION: the deterministic `\_verify()` check (per G1's (b) limitation) only confirms the cited chunk ID EXISTS in the retrieved set — it does NOT check that the chunk's TEXT actually supports the specific numeric claim made, so a claim citing a real, retrieved-but-irrelevant chunk would pass this check even though it shouldn't. To instrument this definitively: log and inspect (1) exactly which chunks were retrieved for these specific adversarial questions (was anything genuinely relevant-looking retrieved, or was it all clearly off-topic?), and (2) the raw claim + cited chunk ID + the actual TEXT of that cited chunk, side by side — if the cited chunk's text plainly does NOT contain the claimed data, that's direct evidence of a synthesis-stage failure (the model fabricated a claim and cited weak/irrelevant evidence anyway), which is the most likely culprit given `\_verify()`'s known limitation (it checks existence, not entailment)

— but this needs to be confirmed by actually reading the retrieved chunk's content, not assumed.

G3. Why does this project's `_verify()` function check citations against the actually-retrieved evidence set rather than trusting the LLM's self-reported claim about which chunk supports which sentence? Give a concrete example of a citation that would pass this deterministic check but still be substantively wrong.

Trusting the LLM's self-report entirely would mean having NO independent check at all — the entire value of `_verify()` is providing a check that doesn't depend on the same model's own (potentially unreliable) judgment about its own output, which is exactly the self-referential problem with approach G1(a). By checking cited chunk IDs against the actual retrieved set, the system catches the specific, common failure of an LLM HALLUCINATING a citation to a chunk that was never even retrieved (a plausible-sounding but entirely fabricated source reference) — a real and common LLM failure mode independent of whether the claim itself is true. Concrete example of what still passes this check despite being wrong: suppose the retrieved evidence includes a real chunk stating "Zions Bank's income before income taxes decreased \$76 million in 2023" (a SEGMENT-level figure, not consolidated), and the model generates the claim "Zions Bancorporation's total consolidated income decreased \$76 million in 2023," citing that real chunk's ID. The chunk ID genuinely exists in the retrieved set, so the deterministic existence check passes — but the claim MISATTRIBUTES a segment-specific figure as a consolidated, company-wide figure, which the cited chunk does not actually support. This is exactly the class of error (real citation, wrong entailment) that `_verify()`'s current design cannot catch, and it's precisely why Q17 in this project's own eval set was specifically designed to probe segment-vs-consolidated confusion — a good illustration of an eval question and a known verification limitation pointing at the exact same underlying risk.

G4. What is RAGAS's "faithfulness" metric actually measuring, in contrast to "context precision" and "context recall"? Give an example answer that would score well on faithfulness but poorly on context recall, and vice versa.

Faithfulness measures whether the GENERATED ANSWER's claims are actually supported by the RETRIEVED CONTEXT — it's entirely about the generation-to-context relationship, and it doesn't care whether the retrieved context was the BEST possible context, only whether the answer is consistent with whatever context it was actually given. Context precision measures how much of the RETRIEVED context is actually relevant/useful (penalizing retrieving a lot of irrelevant noise alongside the useful chunks). Context recall measures whether ALL the relevant information needed to answer the question was actually retrieved in the first place (penalizing missing crucial evidence entirely). Example scoring well on faithfulness but poorly on context recall: retrieval only surfaces a chunk covering HALF of what's needed to fully answer a multi-hop question (e.g., Q20's full quarterly-deposit-trajectory question, if only 2 of the 5 needed filings were retrieved) — the model then generates an answer strictly and accurately based on the partial evidence it received (high faithfulness, no hallucination beyond what was given), but the answer is fundamentally INCOMPLETE relative to the full ground truth, because critical evidence was never retrieved (poor context recall). Example scoring well on context recall but poorly on faithfulness: retrieval successfully surfaces ALL the correct, needed chunks (high recall), but the model's generation still fabricates an unsupported embellishment or misinterprets one of the correctly-retrieved chunks anyway (e.g., the segment-vs-consolidated misattribution from G3) — the retrieval did its job perfectly, but generation introduced an unfaithful claim despite having everything

it needed to answer correctly.

G5. A stakeholder asks: "Why does the system sometimes say 'I don't have enough information' when the answer is clearly somewhere in the corpus?" Using this project's actual architecture, list every layer where that failure could originate, and for each, name one metric from your eval harness that would help localize the fault.

(1) Retrieval layer: the agent's search query, or the underlying embedding/BM25 matching, failed to surface the relevant chunk at all — diagnosed via the retrieval ablation's Hit@k metrics for the specific question/query in isolation (does the chunk appear in top-k when searched directly, bypassing the agent loop entirely?). (2) Agent-loop layer: the model decided to stop searching (hit MAX_SEARCH_ITERATIONS, or judged it had "enough" evidence) before actually finding the relevant chunk, even though a DIFFERENTLY-PHRASED search query would have found it — this is a decision- quality issue distinct from pure retrieval capability, and diagnosing it requires inspecting the actual sequence of search queries the agent chose to issue (logged, not directly reflected in a single aggregate metric — a real gap worth flagging: this project's eval doesn't currently log the agent's intermediate search queries for post-hoc inspection). (3) Synthesis layer: the relevant chunk WAS retrieved and passed to the synthesis step, but the model failed to recognize it as sufficient evidence and generated an unsupported/empty claim anyway — diagnosed by checking, for a specific failing question, whether the correct chunk was actually present in the `evidence` dict passed into `synthesize()` (if yes, the fault is in synthesis, not retrieval or the agent loop). (4) Verification layer: the model DID generate a claim citing the correct chunk, but `\_verify()` incorrectly flagged it as unsupported due to a chunk_id mismatch or formatting inconsistency (e.g., a subtly different chunk_id string that doesn't exactly match) — diagnosed by comparing the raw model output's cited chunk_id string against the exact chunk_id format used in the actual evidence dict, checking for a silent mismatch bug rather than a genuine "model didn't cite" failure.

H. RAG Evaluation Design

H1. Why is "hand-labeling ground truth against real documents" considered more valuable, from an interview-signal perspective, than generating a synthetic golden set entirely with an LLM? What specific failure mode of LLM-generated golden sets does hand-labeling avoid?

An LLM-generated golden set (asking an LLM to both generate questions AND their "correct" answers from a corpus) risks a specific, insidious failure mode: the SAME model family's blind spots and hallucination tendencies can appear on BOTH sides of the eval — the LLM might generate a "ground truth" answer that's subtly wrong (a plausible-sounding but incorrect interpretation of the source text), and then your SYSTEM (which likely uses a similar or related LLM) might independently arrive at that SAME wrong answer, making your eval falsely report high accuracy despite the system actually being wrong relative to the real source document — the eval is only checking self-consistency between two LLM-derived artifacts, not truth. This project's approach — every question's expected answer was checked with a verbatim quote from the actual source .md file (this project's own process specifically upgraded initial "NEEDS VERIFICATION" placeholder answers to fully grep-confirmed, quoted

answers before treating them as ground truth) — grounds every ground-truth answer in the actual document, independent of any LLM's interpretation, which is the only way to guarantee the eval is measuring against REALITY rather than against another model's (possibly shared) blind spots. This is also a stronger interview signal specifically because it demonstrates the discipline of not trusting convenient automation for the one part of your pipeline (ground truth) where correctness matters most — the eval is only as trustworthy as its labels.

H2. Explain precision@k, recall@k, and MRR as used in this project's retrieval ablation. Construct a hypothetical scenario where Hit@5 is high but MRR is low, and explain what that tells you that Hit@5 alone would hide.

Hit@k (this project's specific metric, a simplified precision@k variant for single-relevant-chunk questions) measures whether the correct chunk appears ANYWHERE within the top-k retrieved results — it's a binary, threshold-based measure that doesn't care about exact rank, as long as the answer is "close enough" to the top. MRR (Mean Reciprocal Rank) instead averages $1/\text{rank}$ across all questions — it's rank-SENSITIVE, rewarding the correct chunk appearing at rank 1 much more than at rank 5, and penalizing questions where the correct chunk is present but buried lower in the ranking. Hypothetical scenario: across 10 questions, the correct chunk appears at rank 5 for all 10 (never earlier, never absent) — Hit@5 would be a perfect 100% (the answer IS in the top 5 every time), but MRR would only be $1/5 = 0.20$ (a fairly low score, since the correct answer is never near the top). This combination (high Hit@5, low MRR) tells you something Hit@5 alone completely hides: the system reliably finds the right answer SOMEWHERE in a reasonably-sized window, but its RANKING QUALITY within that window is poor — the correct answer is never confidently placed at the top, meaning a downstream consumer that only looks at the top-1 or top-2 result (rather than examining all 5) would consistently miss it despite the answer technically being "found." This is exactly why this project's eval reports both metrics together rather than either alone — Hit@5 answers "is retrieval capable of finding this at all," while MRR answers the practically more important question "how much can downstream consumers trust the TOP result specifically."

H3. Your retrieval ablation shows each layer helping (14.3% → 28.6% → 32.1% Hit@5). Your project lead says "great, ship it, 32% is fine since it's better than nothing." What is wrong with stopping the investigation there, and what did this project actually do instead when facing an unimpressive absolute number?

The flaw in "each layer helps, therefore we're done" is that it only evaluates RELATIVE improvement between configurations, never asking whether the ABSOLUTE number itself is adequate for the system's actual purpose — a citation-grounded RAG system that only finds the correct source 32% of the time will, by construction, produce unsupported or wrong-cited answers on the majority of real questions, regardless of how much better that number is than a worse baseline. "Better than nothing" is a low bar that says nothing about whether the system is fit for its actual purpose. What this project actually did: rather than accepting 32.1% Hit@5 as a stopping point because it was directionally better than dense-only's 14.3%, it treated the LOW ABSOLUTE NUMBER itself as the signal worth investigating, ran a diagnostic comparing exactly which chunks were retrieved vs. the actual ground truth for specific failing questions, identified a root cause (chunk size causing fact dilution), tested a specific fix (reducing chunk size), and re-measured — resulting in a genuine, MEASURED improvement to 53.6%, not just an assumption that "we did what best practice suggests, so it must be

good enough." The broader principle for an interview: an ablation table's relative ordering tells you your architectural choices are directionally correct, but only comparing the absolute number against your system's actual requirements (here, needing a source found reliably enough for downstream citation to be trustworthy) tells you whether you're actually done.

H4. Why does this project deliberately include "partial coverage" test questions in the eval set, rather than only including questions with fully verified, complete answers? What specific system behavior is this designed to catch?

Questions like "which of the 6 banks had the highest CET1 ratio" (where the eval author only directly verified the CET1 figure for 1 of 6 banks) are deliberately designed to test whether the SYSTEM correctly recognizes and communicates the limits of its own knowledge, rather than confidently asserting a complete-sounding answer built on incomplete evidence. This is a distinct failure mode from simple retrieval misses or citation errors — it's specifically about whether the system HEDGES appropriately when it has SOME but not ALL the information needed for a fully confident claim (e.g., "ranking all 6 banks" implicitly requires data on all 6, and a system that only successfully retrieved data for 2-3 of them should say so explicitly, rather than presenting a 6-way ranking as if it were complete and confident). Without this class of test question, an eval set built entirely from fully-answerable questions would never surface whether a system overclaims completeness — a genuinely important, easily-missed failure mode in real deployments, where users often ask broad questions ("compare all our vendors," "summarize every risk factor") that a RAG system may only have PARTIAL corpus coverage for, and confidently answering as if coverage were complete is arguably a more dangerous failure than simply saying "I don't know," since it looks trustworthy while being silently wrong.

H5. What is the difference between an ablation study and a golden-set eval, and why does this project need BOTH? Specifically, explain why the retrieval ablation could run entirely offline while the citation- verification pass could not.

An ablation study isolates and measures the CONTRIBUTION of individual architectural components (dense-only vs. +hybrid vs. +reranking) against a fixed measurement, answering "which of my design choices actually matter, and by how much." A golden-set eval measures END-TO-END system performance against a fixed set of real questions with known-correct answers, answering "is the whole system, as currently built, actually good." They're complementary, not redundant: the ablation tells you WHY a number is what it is (which layer contributes how much), while the golden-set eval tells you WHAT the number is for the system as a whole, including layers an ablation might not isolate individually (e.g., the citation-verification pass's abstention behavior isn't really an "ablation" of a component — it's an end-to-end behavior of the full pipeline). The retrieval ablation could run entirely offline because Hit@k/MRR only require comparing a RETRIEVED chunk_id against a KNOWN-CORRECT chunk_id — a purely mechanical, local computation needing no LLM call at all, just the embedding/reranking models (which run locally, free, and unlimited) and Qdrant. The citation-verification pass, by contrast, requires an LLM to actually GENERATE an answer with citations (the agent loop's search decisions plus the synthesis step's claim generation) — there is no way to evaluate whether a SYNTHESIZED, LLM-GENERATED citation is correct without actually running the LLM to produce that citation in the first place, making this stage inherently dependent on (quota-limited, rate-limited) Gemini API calls in a way the purely-mechanical retrieval-precision measurement never is.

H6. You have a fixed budget of \$200 in eval-time LLM API calls before a launch deadline. You have 200 golden-set questions but can only afford to run about 60 through the full pipeline. How do you decide which 60 to prioritize, and what's the risk of choosing badly?

Prioritize for COVERAGE ACROSS FAILURE-MODE CATEGORIES rather than random sampling or convenience ordering — this project's own three-bucket design (factual/numeric, multi-hop/cross-reference, adversarial/unanswerable) is exactly the right lens: running 60 questions that are ALL factual/numeric would give you a confident-but-narrow signal (citation accuracy on simple lookups) while leaving you completely blind to whether the system correctly abstains on unanswerable questions (this project's real finding — 0% correct abstention on the 2 adversarial questions actually tested — would NEVER have surfaced if the budget had been spent entirely on factual questions, since those don't test abstention behavior at all). A reasonable allocation: proportionally represent all three buckets (rather than collapsing to just one), with EXTRA weight on the adversarial bucket specifically, since abstention failures (confidently fabricating an answer) are typically higher-STAKES than a citation-accuracy miss on an answerable question — a wrong-but-honest "I don't know" is generally less damaging in production than a confident, plausible-sounding fabrication. The risk of choosing badly (e.g., running only the "easiest to verify" questions, or questions from only one bucket) is that your eval numbers report an overly optimistic picture of a system that's actually failing in an important-but-untested dimension — exactly the situation this project would have been in if the daily-quota-limited partial run had happened to test only answerable questions and never reached the adversarial bucket at all.

I. LLM API Reliability Engineering

I1. Explain the practical difference between a "per-minute rate limit" and a "per-day quota" on a free-tier LLM API, and why the SAME retry-with- exponential-backoff strategy is appropriate for one and actively harmful for the other.

A per-minute rate limit is a SHORT-WINDOW throttle — the underlying capacity to serve your request still exists, it's just temporarily unavailable because you've made too many requests in a brief window; waiting a short time (seconds) and retrying is a legitimate strategy because the constraint genuinely resolves itself quickly. A per-day quota is a HARD CEILING on total requests within a much longer window (a full day) — once exhausted, no amount of waiting SECONDS OR EVEN MINUTES will help, since the constraint doesn't reset until the next day boundary. Applying exponential backoff (this project's `retry.py``: 5s/10s/20s/40s/80s) to a per-minute rate limit is exactly correct — the short waits are proportional to how quickly the constraint actually resolves. Applying the SAME strategy to a daily quota is actively harmful: it burns real wall-clock time (this project measured ~2.5 minutes wasted per occurrence) retrying an error that is GUARANTEED to recur identically on every retry attempt, since nothing about the constraint changes within that timeframe — worse, in an automated pipeline running many sequential calls, this wasted time compounds across every subsequent call that also hits the same exhausted quota, multiplying the wasted time by however many more calls the pipeline was going to make. This project's actual fix — detecting the specific daily-quota error signature and raising immediately via a distinct `DailyQuotaExhausted`` exception rather than retrying — reflects the general

principle that error handling needs to distinguish TRANSIENT from PERMANENT failures, since "retry with backoff" is only a correct strategy for the former.

12. A model alias like "gemini-flash-latest" silently began resolving to a slower, more expensive "thinking" model variant, with no error, warning, or code change on your end. Design a lightweight monitoring approach using only what this project already has, no new infrastructure, that would have caught this automatically.

This project already logs enough raw material to detect this without any new infrastructure — the fix is in HOW that material gets used, not in adding new logging. Specifically: every Gemini API response includes ``usageMetadata`` (visible in this project's own diagnostic curl output, including fields like ``thoughtsTokenCount`` and ``totalTokenCount``) and, critically, response TIMING is already implicitly measurable by wrapping existing calls with a timer. A lightweight approach: add a simple assertion or warning check immediately after each Gemini call in ``agent.py`` and ``synthesis.py`` — if response latency exceeds a threshold (e.g., 3-5 seconds for what should be a fast lite-model call) OR if ``usageMetadata`` contains a non-zero ``thoughtsTokenCount`` (a direct signal the "thinking" variant is being invoked, which this project's diagnostic call revealed as the exact tell-tale sign of the regression), log a clear warning naming the specific model resolved (``response.model_version`` or similar field) rather than just the alias requested. This would have surfaced the exact symptom this project spent real diagnostic time discovering manually (via bare curl calls and response inspection) automatically, the very first time the regression occurred, rather than requiring a 25-minute "why is this hanging" investigation — the general principle being that a system consuming a third-party API by alias (rather than a pinned, dated model version) should actively monitor for silent underlying-model changes, since an alias's whole purpose (auto-updating to "the latest") is also its risk (auto-updating to something meaningfully different in behavior or cost).

13. Why is it dangerous to add a broad `except Exception: continue` around a loop processing untrusted LLM outputs, even though it "fixes" the immediate crash? What did this project do differently to avoid silently masking real bugs while still gaining resilience?

A bare ``except Exception: continue`` (or similar) makes ANY failure — including genuinely serious, systematic bugs that should be investigated and fixed — silently disappear into a "skip and move on" path indistinguishable from a truly expected, benign edge case. This creates a real risk: if a NEW class of bug were introduced later (e.g., a change that broke `chunk_id` formatting across the board, causing every single citation check to fail), a sufficiently broad exception handler would silently "succeed" at running the whole eval to completion while producing completely meaningless results (100% of questions silently erroring and being skipped) — and without deliberately checking, you might not notice the eval effectively evaluated nothing at all. This project's actual approach avoided this by (1) LOGGING every caught exception with its specific type and message (``print(f"Q{qid}: ERROR ({e!r}) — skipping, continuing")``) rather than silently swallowing it, so a human reviewing the run's output can immediately see if failures are rare-and-expected (a handful of transient errors) or systematic-and-alarming (every single question failing identically); and (2) fixing the SPECIFIC, DIAGNOSED root causes (the missing ``query`` argument, the malformed claim structure) with targeted, narrow fixes BEFORE relying on the broad exception handler as a safety net — the broad handler is a last-resort resilience measure for genuinely unpredictable failures, not a substitute for actually diagnosing and fixing the failures you can identify and address specifically.

14. You observe that ~43% of your eval questions are failing with `DeadlineExceeded` errors. Before writing a single line of retry logic, what are three hypotheses you should rule out first, and how would you test each one cheaply?

(1) Client-side hang, not a real server-side timeout: test by making a SINGLE bare API call outside your application code (e.g., directly via curl or a minimal Python script) with a generous timeout, and check whether it actually completes (even slowly) or genuinely never returns — this project did exactly this, discovering the "hang" actually completed in 12 seconds once given enough time, ruling out a true infinite hang and pointing instead at a slow-but-functioning underlying model. (2) Network-level connectivity issue (e.g., broken IPv6 routing, a known real-world gotcha) rather than an application or API-side problem: test with ``curl -4`` vs ``curl -6`` explicitly to isolate whether one IP version fails while the other succeeds — this project ran exactly this test and found IPv6 failing instantly while IPv4 worked, which initially looked like the culprit before further testing (a longer-timeout retry) revealed the real cause was elsewhere (a slow model, not a hung connection) — an important lesson that an early, plausible-looking hypothesis can be WRONG even when a supporting test result seems to confirm it, and further testing is warranted before concluding causation. (3) The specific timeout value configured client-side being too aggressive for genuine (but bounded) API latency variance, rather than any external factor at all: test by re-running a sample of the failing calls with a substantially longer timeout and checking whether they now succeed — if a 30-second timeout consistently fails but a 60-second timeout consistently succeeds, that's evidence the timeout budget itself, not any deeper issue, was too tight for real, bounded API latency variance under load.

15. This project's `retry.py` treats `ResourceExhausted` (rate limit) and `DeadlineExceeded` (gateway timeout) as retryable, but `DailyQuotaExhausted` as immediately fatal. What general principle determines whether an error should be retried, and how do you apply that principle to a new, previously-unseen error type you haven't hard-coded a rule for?

The general principle: an error should be retried if and only if there's a plausible mechanism by which the SAME request, sent again after a short wait, could succeed where it just failed — i.e., the failure's underlying cause is TIME-BOUNDED and likely to resolve on its own (transient network issues, momentary server overload, short-window rate limits). An error should NOT be retried if the failure's cause is a HARD, TIME-INDEPENDENT constraint that a short wait cannot possibly resolve (an exhausted daily quota, an invalid API key, a malformed request that will fail identically every time). Applying this to a genuinely new, unclassified error type: the safe default is to NOT blindly retry it as if it were transient — instead, inspect the error's message/metadata for signals of its TIME CHARACTER (does it mention a specific reset time or "retry-after" duration, suggesting transience? does it reference a fixed resource limit with no time dimension, suggesting permanence?), and if genuinely ambiguous, err toward treating it as non-retryable initially (failing fast and surfacing it for human investigation) rather than risking the `DailyQuotaExhausted`-style mistake of retrying something un-resolvable and wasting real time — a conservative default that costs a bit of missed transient-error recovery in exchange for never repeating the "wasted 2.5 minutes retrying an unrecoverable error" mistake this project actually encountered before fixing it.

J. Cost & Latency Trade-offs

J1. In an agentic loop that may call an LLM 2-6 times per user question, why does per-call latency matter disproportionately more than in a single-call RAG system? Do the math: at 12 seconds/call versus 1 second/call, what's the user-facing latency difference for a question requiring 4 tool-call round-trips plus 1 synthesis call?

In a single-call RAG system, per-call latency directly IS the user-facing latency — there's a 1:1 relationship. In a multi-call agentic loop, per-call latency gets MULTIPLIED by the number of sequential calls the loop makes, since each call in this project's hand-rolled loop depends on the result of the previous one (the model needs the prior search result before deciding its next action) — these calls cannot be parallelized, they're inherently sequential. Doing the math for the described scenario (4 tool-call round-trips + 1 synthesis call = 5 total sequential Gemini calls): at 12 seconds/call (the "thinking" model variant this project actually hit), total user-facing latency would be $5 \times 12\text{s} = 60$ seconds — a genuinely unacceptable wait for an interactive research-assistant use case. At 1 second/call (the "lite" model variant this project switched to), the same 5-call sequence totals just 5 seconds — a 12x improvement in absolute user-facing latency, purely from model selection, with no change to the agent loop's logic or architecture at all. This concretely demonstrates why per-call latency in a multi-call pipeline isn't just "somewhat more important" than in a single-call system — it's a MULTIPLICATIVE factor, meaning a seemingly modest per-call latency difference compounds into a dramatic, potentially product-breaking difference in the end-to-end experience.

J2. You need to choose between a fast/cheap model for an agent's tool-calling loop and a slower/higher-quality model for final answer synthesis. Justify using two DIFFERENT models for these two roles within the same pipeline, and identify one risk this two-model approach introduces.

Justification: the tool-calling loop's job (deciding whether/what to search, per ADR-005) is a relatively narrow, mechanical decision — "do I have enough evidence, and if not, what should I search for next" — a task that doesn't require deep reasoning quality, and where LATENCY compounds multiplicatively across iterations (per J1), making speed the dominant concern for this specific role. The final synthesis step, by contrast, is a ONE-TIME call per question (no compounding latency cost) where the actual OUTPUT QUALITY — correctly reasoning about which claims are genuinely supported by evidence, correctly distinguishing consolidated vs. segment-level figures (per G3), and producing well-structured, accurate citations — matters far more than shaving off another second of latency, since there's no multiplicative penalty for this single call being somewhat slower. Using a faster/cheaper model for the loop and a more capable model for synthesis lets you optimize each stage for what it actually needs, rather than picking one model that's a compromise for both roles. The risk this introduces:

INCONSISTENCY in behavior/capability between the two stages could create subtle bugs — e.g., if the fast tool-calling model retrieves evidence based on its own (weaker) judgment of "have I searched enough," but the synthesis model (with stronger reasoning) would have wanted DIFFERENT or MORE evidence to answer well, there's a capability mismatch where the weaker model's decisions bottleneck the stronger model's output quality — the synthesis model can only be as good as the evidence handed to it, regardless of its own superior reasoning ability, so this two-model split only pays off if the cheaper model is still competent enough at its specific narrow task (deciding what to search for), not just cheap.

J3. What would "cost per 1,000 user questions" look like for this project's architecture, and what's the first thing that breaks if usage grew 100x? Is it the Gemini quota, the local compute, or something else?

Current architecture cost breakdown: local embedding, local reranking, and Qdrant (self-hosted via Docker) are all \$0 marginal cost per question — the only per-question cost is Gemini API usage in the agent loop and synthesis step, which is currently on the FREE TIER, meaning "cost per 1,000 questions" today is \$0 in dollar terms but bounded by a hard DAILY REQUEST QUOTA rather than a dollar budget. At 100x usage growth, the Gemini daily quota would be the FIRST thing to break, and it would break almost immediately — this project already demonstrated exhausting a free-tier daily quota with fewer than 40 evaluation questions in a single session (each question consuming multiple sequential Gemini calls); 100x realistic production usage would exhaust any free-tier daily allowance within a small fraction of a single day, long before local compute (embedding/reranking, which scales with CPU/GPU capacity you control and can scale horizontally) or Qdrant (which can be vertically or horizontally scaled per ADR-001's own consequences section) become the binding constraint. This is a genuinely different SHAPE of bottleneck than a typical infrastructure scaling problem: it's not "our servers can't handle the load," it's "our chosen PRICING TIER has a hard ceiling regardless of infrastructure capacity" — the practical first fix at scale isn't more compute, it's moving off the Gemini free tier onto a paid tier (or a different provider/self-hosted model) entirely, a business/cost decision as much as a technical one.

K. Financial Document Domain Knowledge

K1. Explain what inline XBRL is and why SEC filings (post-2019 mandate) contain a large block of machine-readable tagging data interleaved with human-readable text. What specifically goes wrong if a naive HTML-to-text scraper doesn't account for this?

Inline XBRL (iXBRL) is a format that embeds machine-readable financial data tags (identifying specific numbers as, e.g., "TotalDeposits" or "NetIncomeLoss" per a standardized taxonomy) directly within the same HTML document that displays the human-readable filing — this lets regulators and analysts programmatically extract structured financial data without needing a separate machine-readable file, but it means the underlying HTML contains BOTH the visible, formatted filing text AND large blocks of hidden (`display:none`` styled) or specially-tagged (`<ix:header>``) metadata that isn't meant to be visually displayed to a human reader viewing the document in a browser. This project encountered this concretely: a naive BeautifulSoup-based text extraction, without specifically stripping the `<ix:header>`` block and any `display:none`` styled elements, produced output that was ENTIRELY XBRL metadata garbage (raw taxonomy references like "us-gaap:CommonStockMember," dates, and namespace URLs) rather than the actual readable filing prose — the fix required explicitly detecting and removing these specific structural patterns (a regex stripping the `<ix:header>`` block, plus decomposing any element with a `display:none`` style) before extracting visible text, a domain-specific parsing detail that wouldn't be obvious from generic HTML-scraping experience, and that would silently corrupt an entire ingested corpus if missed — the resulting "chunks" would all be unusable metadata noise rather than actual filing content, a failure that might not even be obvious until inspecting the actual chunked output.

K2. A user asks your system "What was Company X's total deposits?" without specifying a date or filing period. Given that SEC filings routinely restate the SAME historical figure differently across different filings, what does a well-designed RAG system need to do differently here compared to answering a question about a static fact?

A question without a specified time period is fundamentally UNDERSPECIFIED against a corpus containing multiple filings for the same company across different periods — this project's own eval design deliberately surfaced this exact ambiguity (e.g., Q18's WAL uninsured-deposit question requires connecting a dollar figure disclosed in one fiscal year's 10-K to a RATIO disclosed for the SAME historical date in a LATER 10-K, since the same fact can be framed differently depending on which filing you're reading). A well-designed system needs to either (a) ask a CLARIFYING question back to the user rather than silently picking one period's figure and presenting it as THE answer (the current answer, when there are multiple valid, period-specific answers, is genuinely ambiguous rather than simply "missing"), or (b) if answering directly, explicitly STATE which filing/ period the figure comes from as part of the answer, rather than presenting a bare number as if it were a single, timeless fact — since presenting "\$55.3 billion" without specifying "as of December 31, 2023, per the FY2023 10-K" would be actively misleading given that the SAME company's deposits were \$47.6 billion just one quarter earlier during the 2023 banking-stress trough (this project's own real quarterly data). This is a genuinely different challenge than answering a question about a static, unchanging fact (e.g., "what state is the company headquartered in"), where any retrieved answer is likely correct regardless of which filing it came from — the system needs a mechanism (either in the agent loop's query formulation, or in synthesis's answer framing) to recognize which questions are period-sensitive and handle the ambiguity explicitly rather than silently.

K3. Why is "regional banks in 2023" specifically a harder, more differentiated corpus for a RAG eval than "clean, well-structured single-topic documents," in terms of the actual retrieval/citation challenges it creates?

This corpus creates several genuinely hard, compounding challenges beyond just "financial jargon is hard": (1) Tables that break naive chunking — each 10-K contains roughly 150-170 raw HTML tables, and dense financial data tables don't split cleanly along the sentence/paragraph boundaries most chunking strategies assume, requiring explicit handling (this project's pipe-delimited table rendering) rather than defaulting to prose-oriented recursive splitting. (2) Genuine version drift with a real historical event driving it — the same company's SAME metric (e.g., WAL's insured deposit ratio) has meaningfully DIFFERENT, both-correct answers depending on exactly which historical date and which filing you're asking about, driven by an actual, dramatic real-world event (the March 2023 regional banking stress) rather than an artificial or contrived scenario — this creates GENUINE ambiguity that a RAG system must handle carefully (per K2), not a toy problem. (3) Structurally-templated boilerplate across DIFFERENT facts within the same document (per D6) — a failure mode that stresses BOTH dense and sparse retrieval simultaneously in a way clean, single-topic prose never would, since clean prose rarely repeats near-identical sentence templates across many different underlying facts. (4) Cross-company, same-sector terminology collision — because all 6 companies in this project's corpus operate in the same regional-banking sector, generic financial terms (deposits, CET1 ratio, provision for credit losses) appear across ALL of them, meaning retrieval must correctly disambiguate WHICH company's specific figure is relevant to a query, not just find "a" chunk about deposits generally — a genuinely harder retrieval problem than a corpus where each document covers an entirely distinct topic

with little cross-document vocabulary overlap.

L. Capstone System Design Scenarios

L1. You're asked to redesign this Research Copilot to handle 10,000 documents (500x the current corpus) and 1,000 concurrent users, while keeping the \$0/month local-inference cost model. Walk through which THREE components would break first, in order, and what you'd change for each.

(1) FIRST to break: Gemini's free-tier daily quota — this is a hard constraint completely independent of document/corpus size, driven purely by CONCURRENT USER VOLUME and the number of Gemini calls per question (per J1/J3); at 1,000 concurrent users, even a handful of questions per user would exhaust any free-tier daily allowance almost immediately, likely within minutes of real traffic, making this the very first and most severe bottleneck — the fix here isn't architectural at all, it's a business/cost decision to move off the free tier onto a paid tier (breaking the stated \$0/month constraint) or replace Gemini with a self-hosted open model for the agent loop and synthesis, which reintroduces real infrastructure cost and complexity but removes the hard quota ceiling. (2) SECOND to break, assuming (1) is solved: local CPU-bound embedding/ reranking throughput under concurrent load — this project's local models (bge-base embedding, bge-reranker cross-encoder) run fine for sequential, single-user development and ingestion, but 1,000 CONCURRENT users each triggering real-time query-time embedding and reranking would saturate CPU capacity on a single machine quickly; the fix is either GPU acceleration (genuinely violating the "local, free" framing if GPU infrastructure has real cost) or horizontal scaling of the embedding/reranking service behind a load balancer, introducing real infrastructure complexity that didn't exist at the original single-developer scale. (3) THIRD to break: Qdrant's single-node HNSW index at 10,000 documents' worth of chunks — per A5's diagnostic framework, this is likely still manageable via quantization before needing sharding, since even 500x this project's current ~17,000 chunks (roughly 8.5 million chunks) is within range for a well-tuned single Qdrant node with quantization, meaning this is genuinely the LEAST urgent of the three, contrary to a naive assumption that "more documents" is the primary scaling axis to worry about — concurrent USER volume, not document COUNT, is the dominant driver of what breaks first in this specific scenario.

L2. A stakeholder asks you to add support for legal contracts to the SAME system, reusing as much infrastructure as possible. Which existing ADR decisions would need to be REVISITED for this new domain, and which would likely transfer unchanged?

LIKELY TRANSFERS UNCHANGED: the vector store choice (ADR-001, Qdrant) — nothing about legal contracts specifically demands a different vector database technology; hybrid search (dense+sparse fusion) remains equally valuable for contracts, which also mix precise defined-term lookups (BM25-favorable) with conceptual/semantic queries (dense-favorable). The agent loop architecture (ADR-005, hand-rolled tool-calling) also transfers directly — the mechanics of "decide whether/what to search" are domain-agnostic. LIKELY NEEDS REVISITING: the embedding model (ADR-002) — `bge-base-en-v1.5` was never validated against legal language specifically (per B5's domain-mismatch discussion, the same reasoning applies to legal jargon and defined-term conventions

as it did to financial jargon), and a legal-domain-aware embedding model might be worth evaluating given the high stakes of contract misinterpretation. The chunking strategy (ADR-003) almost CERTAINLY needs revisiting — this project's empirically-tuned 200-token chunk size was validated specifically against dense, templated financial filing text; legal contracts have entirely different structural properties (numbered clauses, cross-references between sections, defined-terms sections that apply throughout a document) that likely demand different chunk boundaries and possibly a stronger case for Parent-Child chunking (D3), since legal clauses frequently reference OTHER clauses by number, creating exactly the kind of context-loss-on-isolation problem Parent-Child chunking targets. The citation-verification approach (ADR-006) would need its BAR RAISED, not fundamentally redesigned — legal contract analysis is inherently higher-stakes than financial-filing Q&A, so the tolerance for imperfect citation (this project's real 60.7% answerable accuracy) that might be acceptable for a research-assistant tool would likely be unacceptable for a legal-analysis tool, meaning the post-hoc NLI/embedding attribution layer (G1's option (c), currently deferred in this project) would become a much stronger candidate for actual implementation rather than remaining a documented-but-unbuilt future option.

L3. Your citation-verification eval revealed a 0% abstention rate on 2 tested adversarial questions. Your manager wants this fixed before the two remaining weeks end, alongside three other roughly-equal-priority items (finishing the eval set, adding the UI, building the semantic cache). Argue for where this fix should rank in priority.

This fix should rank FIRST, above all three other items, and the argument rests on the difference between a QUALITY gap and a TRUST/SAFETY gap. Finishing the eval set, adding a UI, and building a semantic cache are all genuinely valuable, but they're all "make a working system better/more complete" work — none of them address a scenario where the system actively MISLEADS a user with a confident, well-formatted, seemingly well-cited answer that is entirely fabricated. A hallucination that LOOKS well-supported (complete with a citation marker) is arguably worse than an obviously broken feature, because it erodes the entire premise the system is built on — every other citation the system produces becomes suspect once a user discovers even one confidently-fabricated one, since the whole value proposition (per this project's own framing) is "cited, synthesized answers you can trust, unlike a plain LLM." A missing UI is an obvious, visible gap users can see and route around; a fabricated citation is an invisible, silent failure a user has no way to detect without independently verifying the source themselves — which defeats the entire purpose of having citations in the first place. Additionally, per G2's diagnostic framing, this specific failure (system fails to abstain on image-only, genuinely-unanswerable-from-text questions) likely has a narrow, findable root cause (retrieval surfacing weak-but-plausible evidence, or synthesis over-trusting it) rather than requiring a large architectural overhaul — meaning it's plausibly a HIGH-URGENCY, LOW-EFFORT fix once properly diagnosed, making the case for prioritizing it even stronger, since it's not obviously trading off large amounts of time against the other three items.

L4. Design an experiment (not just "run RAGAS") to determine whether your reranker is HELPING or HURTING at your current chunk size, given the ambiguous MRR/Hit@1 vs Hit@5 result from this project. What additional data would you need that the current ablation doesn't already give you?

The current ablation only reports AGGREGATE metrics (Hit@1, Hit@5, MRR averaged across all 28 questions) — what's missing is PER-QUESTION, PRE-AND-POST-RERANK rank data: for every question, the exact rank of the ground-truth chunk BEFORE reranking (in the raw hybrid-search order) versus AFTER reranking. With this per-question data, you can directly categorize every question into one of four buckets: (a) reranking IMPROVED the correct chunk's rank, (b) reranking WORSENEDED the correct chunk's rank but it remained in top-5, (c) reranking WORSENEDED the correct chunk's rank enough to fall OUT of top-5 entirely, (d) reranking had no effect (chunk wasn't in the top-20 candidate set either way, so reranking couldn't help). This categorization directly resolves the ambiguity: if bucket (b) is large (reranking demotes already-well-ranked correct chunks without pushing them out of the useful top-5 window), that explains a Hit@1/MRR decline alongside a stable-or-improved Hit@5 — consistent with E2's Hypothesis 1 (a reranker judgment-quality issue on already-decent candidates, not a coverage problem). If bucket (a) or "new chunks pulled into top-5 from outside top-20-but-still-relevant" is large instead, that would point to a genuinely different mechanism. Beyond per-question rank deltas, it's also worth manually reading a sample of specific (b)-bucket cases — for each, comparing the query text, the correct chunk's content, and whatever chunk the reranker promoted ABOVE it — to build a qualitative sense of WHAT KIND of content the reranker seems to systematically (mis)prefer, which aggregate metrics alone can never reveal, since two numbers (before/after MRR) can be consistent with many different underlying reranker behaviors.